Seatline: Design and Technical Report

Scalable Services SEZG583, MTech, Assignment 1. Optional challenge: 3C Saga with compensation. Team names, institute identifiers, repository and video links: complete the submission appendix. This report describes implemented behavior, not a production certification.

1. Problem and Scope

Seatline books general-admission places for small events. It is deliberately not a CRUD catalog: the main problem is safely allocating finite seats while handling duplicate booking requests and partial failures across independent business capabilities.

Assumptions: one currency, integer minor units, one event per booking, at most ten guests, synthetic contact addresses, and trusted local callers. Event capacity is fixed after creation. A booking is confirmed only after inventory, payment and delivery succeed. Payment and notification are our own mocks; no bank, email or SMS provider is contacted. Authentication, seat selection, real payment integration and production UI are out of scope.

BR1: accepted reservations must never exceed capacity. BR2: release is idempotent and late reservations cannot reverse cancellation. BR3: a declined payment never becomes CHARGED. BR4: refund can be repeated without a second monetary effect. BR5: CONFIRMED is terminal. BR6: failed notification is never recorded as delivered. BR7: the same request/key produces an identical response and only one effect; a different payload conflicts. BR8: failure at saga step three refunds payment and releases seats.

The README maps every rule to its implementing symbol and named pytest check. BR1-BR6 have direct domain tests, with PostgreSQL-backed tests where transactional behavior matters. BR7 and BR8 have live HTTP integration tests because their contracts cross boundaries.

2. Service Decomposition

Inventory owns capacity, reservation holds and cancellation fences. Booking owns the customer-facing lifecycle, request payload, state transitions and recovery. Payment owns mock financial outcomes and refund/cancellation fences. Notification owns mock delivery outcomes and provider rejection. Every capability has its own process, dependency file, environment template, OpenAPI document and database credentials.

Boundaries follow business capabilities, not controller/database layers. Payment and notification do not know inventory internals and can be reused by a different booking workflow. Inventory can serve direct capacity requests independently. Booking is an orchestrator, not a proxy that reads other databases. The common Python module contains only logging, errors, request replay and database setup. It contains no shared entities.

Rejected alternatives: one shared schema would make atomic checkout easier but violate data ownership; a service per table would add network boundaries without independent business responsibilities; merging mock providers into Booking would hide the required partial-failure behavior. Four capabilities are enough to demonstrate the problem.

3. Architecture

The client calls Booking over versioned HTTP JSON. Booking calls Inventory, Payment and Notification directly. There is no A-to-B-to-C-to-D service chain: the longest causal network path has client, orchestrator and one participant. Three participant calls are sequential because the authoritative price follows reservation and delivery follows successful payment. This adds latency but keeps the saga understandable.

All outgoing calls propagate X-Request-ID. Logs are JSON objects with service, method, path, status, elapsed time and request ID. A bounded outbound timeout prevents an unresponsive mock from blocking forever. GET /health checks the service's own database, not all downstream services; one failed participant does not conceal healthy peers.

Happy path: PENDING -> RESERVED -> PAID -> CONFIRMED. Failure path: a dependency failure enters COMPENSATING, refunds/cancels the mock charge, and releases/cancels the hold. Acknowledged compensation ends in CANCELLED. Failed compensation persists COMPENSATION_PENDING until an operator requests recovery. CONFIRMED and CANCELLED are terminal. A PostgreSQL advisory lock serializes work on a booking across workers. Editable component, sequence, state and ER diagrams are in docs/diagrams.md. Rendered component, sequence and per-service data diagrams are included in this PDF.

4. Data Design

Inventory database: events(id, name, capacity, available, price_cents), holds(id, event_id, quantity, state), released_holds(id), idempotency(key, fingerprint, status, response). The only inventory foreign key is holds.event_id -> events.id, within the same database. A CHECK constrains available to zero through capacity.

Booking database: bookings(id, payload, state, amount_cents, reason), transitions(sequence, booking_id, state, created_at), idempotency. The payload stores the request for recovery; its event ID is an opaque remote reference. The local transitions FK references bookings. Payment database: payments(booking_id, amount_cents, state), idempotency. Notification database: deliveries(booking_id, recipient, state), idempotency. Remote booking IDs are not foreign keys and are never joined to another database.

There are four PostgreSQL databases with four distinct login roles. CONNECT is revoked from PUBLIC and granted only to each database's owner. Separate database servers are not needed to demonstrate ownership; sharing a server does share a failure domain. Only offline assessment scripts inspect multiple databases, authenticating separately. Services never receive another service's database credentials.

5. API Design

Booking: GET /api/v1/events aggregates the inventory API; POST /api/v1/bookings creates a booking; GET /api/v1/bookings and /bookings/{id} inspect state; GET /api/v1/bookings/{id}/transitions gives history; POST /bookings/{id}/recover resumes work. Inventory: GET/POST /api/v1/events; POST /api/v1/holds; GET /holds/{id}; POST /holds/{id}/release. Payment: POST /api/v1/charges; GET /charges/{booking_id}; POST /charges/{booking_id}/refund. Notification: POST /api/v1/deliveries; GET /deliveries/{booking_id}.

Service	Method	Path (prefix /api/v1)	Main outcome
Booking	GET	/events; /bookings; /bookings/{id}	200 current representation
Booking	POST	/bookings	201 confirmed; 409 cancelled; 202 pending
Booking	GET	/bookings/{id}/transitions	200 ordered audit history
Booking	POST	/bookings/{id}/recover	200 current recovered state
Inventory	GET, POST	/events	200 list; 201 created; 409 conflict
Inventory	POST	/holds	201 reserved; 409 insufficient capacity
Inventory	GET	/holds/{id}	200 hold; 404 missing
Inventory	POST	/holds/{id}/release	200 released or cancellation fenced
Payment	POST	/charges	201 mock outcome; 409 existing/cancelled
Payment	GET	/charges/{booking_id}	200 payment; 404 missing
Payment	POST	/charges/{booking_id}/refund	200 refunded or cancellation fenced
Notification	POST	/deliveries	201 delivered; 503 mock failure
Notification	GET	/deliveries/{booking_id}	200 delivery; 404 missing

Every service exposes GET /health plus /docs and /openapi.json. Exported specs are under docs/openapi/{booking,inventory,payment,notification}.json. Pydantic validates positive quantities, bounded lengths, amounts, and mock-delay ranges. The contact field is a bounded recipient identifier; browser email validation is not a backend identity guarantee.

Create requests require Idempotency-Key. X-Request-ID is accepted or generated and returned. A sample booking body is {"event_id":"jazz-night","quantity":1, "recipient":"student@example.test"}. Successful creation returns HTTP 201 and a booking with state CONFIRMED. A compensated booking returns 409 with state CANCELLED; incomplete compensation returns 202. These are domain outcomes with a booking representation. Validation errors use 422; missing resources use 404; payload conflicts use 409; mock provider failure uses 503. API errors use {"error":{"code":"...","message":"..."}}. The CANCELLED booking representation also includes this error object. The booking outcome retains its ID so callers can inspect or recover it.

Canonical JSON of validated input is SHA-256 fingerprinted. Endpoint-scoped key, fingerprint, original HTTP status and response body are persisted together. Replays return the original status/body and Idempotency-Replayed: true; correlation IDs may differ. Defaulted values are normalized before fingerprinting, and a mismatched payload is rejected, including after a crash before the final response was stored.

6. Engineering Challenges

Challenge 1, concurrency: a single conditional UPDATE decrements available only when available >= requested quantity. PostgreSQL serializes conflicting row updates; the predicate is evaluated against the current row after waiting. Hold insertion and the decrement share a transaction. A database CHECK provides a second invariant. A plain application SELECT followed by UPDATE would let two callers both observe the same last seat, then both accept it. Process-local locks would not protect separate workers.

The repeatable script synchronizes 20 HTTP callers with a barrier against capacity 5. Expected and recorded outcome: five 201 results, fifteen meaningful 409 errors, and zero available seats with five active holds. Evidence: challenge-1-concurrency/result.json, database-state.json and service-logs.txt. Row locking with SELECT FOR UPDATE was a valid alternative; the conditional update needs less application state and fewer statements.

Challenge 2, idempotency: an endpoint/key-scoped transaction advisory lock serializes identical concurrent requests before checking the durable replay table. Independent keys proceed concurrently. Side-effecting participant changes and their replay rows commit atomically. Booking commits saga checkpoints independently so a crash does not lose remote progress; a deterministic booking ID derived from the key lets a retry resume rather than create another booking. A second per-booking lock also serializes explicit recovery with creation.

The script sends two identical booking requests simultaneously, replays sequentially, and then changes quantity with the same key. Evidence shows identical 201 bodies, 409 conflict, one booking, one CHARGED payment, one hold, persisted fingerprint and serialized response. Logs include idempotency_replay. The original create response does not change after recovery; GET returns the current booking. No key TTL is used.

Challenge 3C, saga: force notification rejection after RESERVED and PAID are durable. The orchestrator enters COMPENSATING and explicitly calls refund then release. The script proves transitions RESERVED, PAID, COMPENSATING, CANCELLED, payment REFUNDED, hold RELEASED, and restored capacity. Recovery on the terminal booking does nothing. This is a real partial failure, not a rollback of one shared SQL transaction.

Compensation endpoints are locally idempotent. Cancellation markers fence off delayed requests: release-before-reserve prevents a late hold, and refund-before-charge creates a CANCELLED payment marker. Per-entity advisory locks order racing forward/compensating operations. If a participant remains unavailable, no false consistency is claimed: COMPENSATION_PENDING is durable and needs explicit recovery. Challenge 3A is not claimed; there is no automatic exponential retry or circuit breaker.

7. Testing

Pytest covers pure state/payment/notification rules, actual PostgreSQL inventory transactions, and live HTTP workflows. The core concurrency test uses 20 database connections; the HTTP challenge also uses a barrier and 20

parallel callers. There are fourteen passing tests in the final suite, including error contracts, correlation ID propagation, payment-decline rollback and cancellation-before-forward-request checks. The committed JUnit/text evidence is the source of truth for the count and timestamp. Tests use unique fixture IDs and never truncate application data.

Evidence includes the normal four-service booking, over-capacity rejection, simultaneous duplicate creation, changed-payload rejection, and third-step failure with compensation. The artifact exporter captures independent database snapshots, structured raw logs, and asserts that each login has CONNECT permission only to its own application database. The UI receives additional browser checks at desktop/mobile widths; the API tests are the grading-critical checks. Run commands and environment variables are in README.md.

Limit of evidence: this is a local demonstration, not a load certification. Connection pools and thread pools bound throughput. Process-kill recovery needs an operator; the complete space of network partitions and crash timings has not been exhaustively tested. Do not treat local tests as a proof of production-grade exactly-once delivery.

8. Design Decisions and Trade-offs

Decision 1: synchronous REST vs broker events. REST matches the assignment and gives easy-to-trace failure demonstrations. A broker would improve decoupling and catch-up but introduces an outbox, consumers and another operational component. It is deferred.

Decision 2: orchestration vs choreography. Booking owns a short explicit state machine, which makes compensation and a student demo easier to explain. The orchestrator is an availability dependency; choreography would distribute ownership but obscure recovery.

Decision 3: PostgreSQL atomic update vs optimistic application checks. The atomic update is correct across processes with simple SQL. SQLite or a Python mutex would make local setup lighter but would not meet distinct database credentials or multi-worker semantics.

Decision 4: notification as saga step vs eventual notification. Here delivery is defined as a prerequisite for confirmation so a step-three failure can compensate two completed steps. A real ticketing system should usually confirm payment and queue notification, not cancel a sale because email is unavailable. The educational policy is explicit.

Decision 5: shared infrastructure library vs copied boilerplate. A small versioned local module keeps errors/replay/logging consistent without sharing business entities. It is included in each service image, so services deploy independently but changes to its contract require coordinated compatibility review.

9. Limitations and Future Work

No authentication, authorization, real money, SMTP, inventory admin security, automatic hold expiry, durable queue, automated compensation scheduler, schema migration framework, or multi-region strategy. Creation uses startup DDL for a teaching-sized project. API collections are unpaginated. The mock notification cannot unsend a message after an ambiguous acknowledgement; compensation guarantees money/capacity consistency, not reversal of external communications. Audit retention and idempotency pruning are deferred.

Per-key and per-booking locks occupy database connections during the synchronous saga. Duplicate storms can exhaust bounded pools. A production design would use durable work claiming and asynchronous recovery. Pricing is read from the inventory hold, never trusted from the caller. Contact details and keys are synthetic; production privacy and secret rotation are separate requirements. External photos/fonts are optional UI assets.

Dockerfile and Compose prepare Assignment 2, with a non-root application user and environment-based endpoints. Minikube deployment, Kubernetes Secrets, readiness/liveness split, production migrations, metrics, and scheduled recovery remain future work.

10. Submission Links

GitHub repository: TO BE COMPLETED BY TEAM. Google Drive folder containing video and other documents: TO BE COMPLETED BY TEAM. Demo video (8-10 minutes, every member speaking): TO BE RECORDED AND

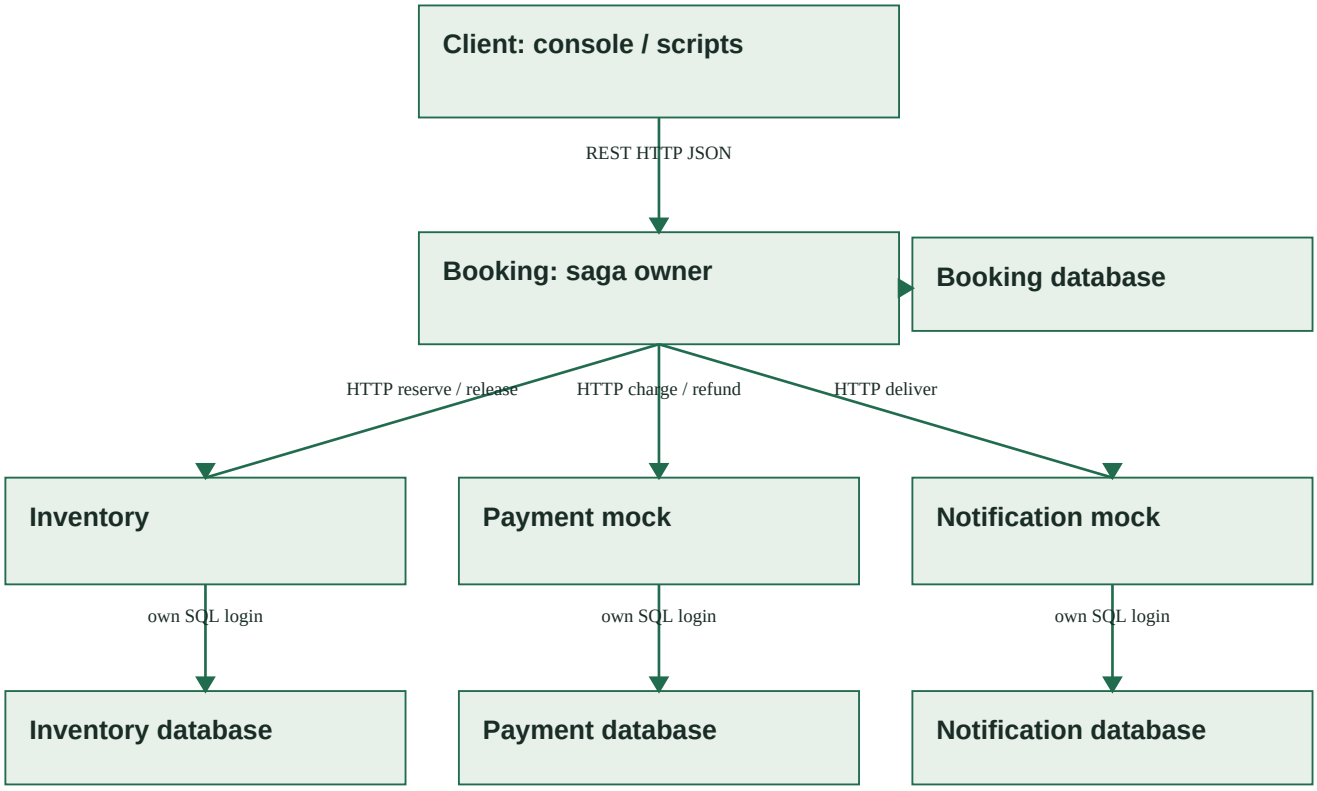
LINKED. Do not submit placeholder links. docs/submission.md contains the speaking outline, contribution table, individual reflection prompts and AI-use declaration.

Appendix: Contributions, Reflection and AI Use

Replace the contribution template with real names, ownership, review contributions and actual commit hashes. Do not invent commits or evenly assign credit without evidence. Each student writes their own half-page reflection: work built, hardest problem solved, how correctness was checked, and what they would change.

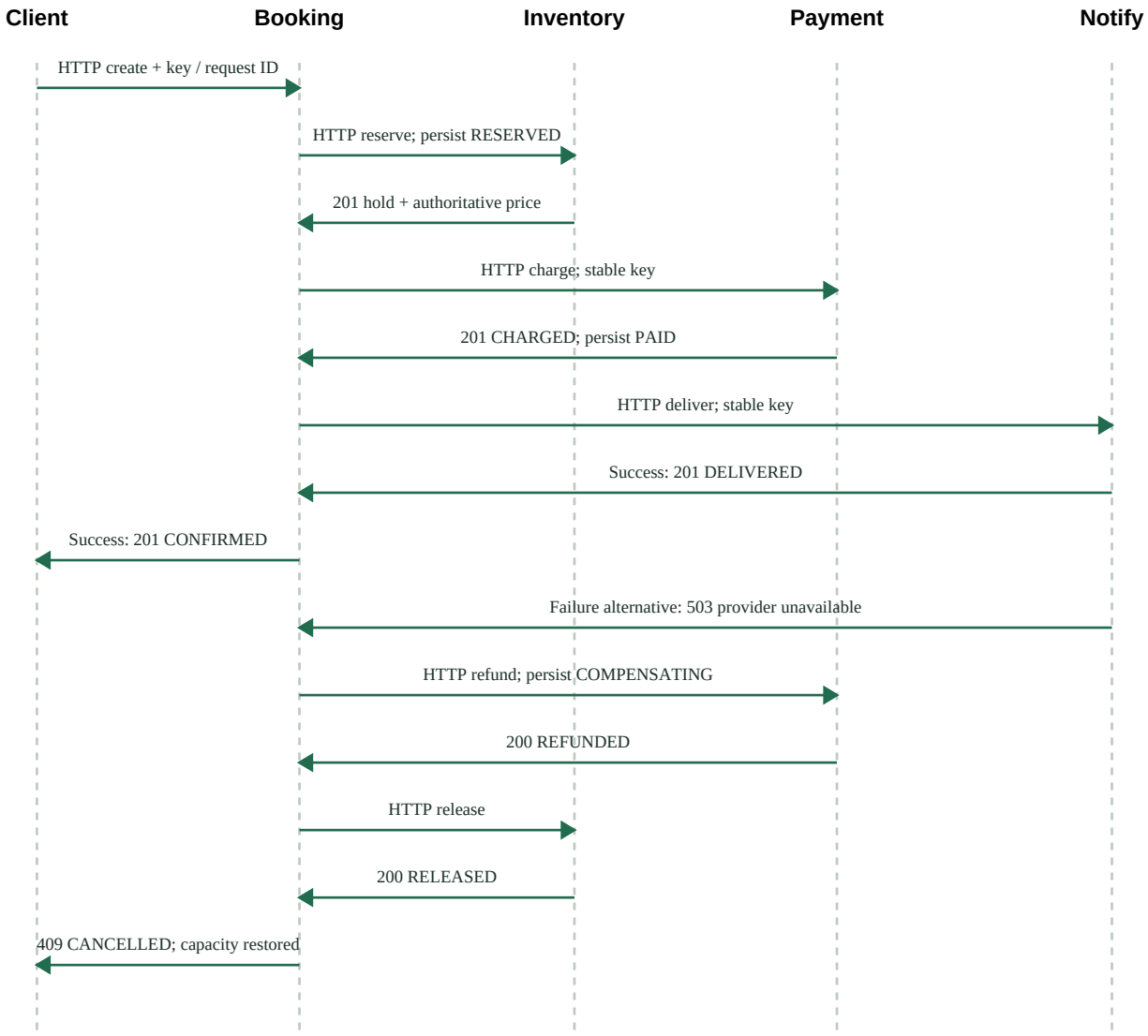
AI declaration draft: GitHub Copilot assisted with application scaffolding, Python and JavaScript implementation, pytest checks, documentation drafts and local execution. The team must review, understand, adapt and validate all generated work, list any other tools used, and explain the atomic SQL, advisory locks, fingerprint storage, and saga failure path independently in the demo. This is a starting declaration, not a claim that students have already performed that review.

Architecture: Component Diagram



Arrows show direction and protocol. Each participant owns a database and a distinct login.

Architecture: Happy and Failure Sequences



Success and failure are alternatives after notification. Compensation reverses the two completed steps.

Data: Per-Service ER and Ownership

Inventory DB / inventory_user

events: PK id, name, capacity, available, price_cents
holds: PK id, FK event_id, quantity, state
events 1 --> many holds (local FK)
released_holds: PK id (cancellation fence)
idempotency: PK key, fingerprint, status, response
CHECK 0 <= available <= capacity
All updates and holds commit atomically.

Booking DB / booking_user

bookings: PK id, payload, state, amount_cents, reason
transitions: PK sequence, FK booking_id,
state, created_at
bookings 1 --> many transitions (local FK)
idempotency: PK key, fingerprint, status, response
payload includes opaque event ID.

Payment DB / payment_user

payments: PK booking_id, amount_cents, state
idempotency: PK key, fingerprint, status, response
booking_id is a remote ID, not a foreign key.
CHARGED / DECLINED / REFUNDED / CANCELLED
Cancellation fence prevents delayed charge.

Notification DB / notification_user

deliveries: PK booking_id, recipient, state
idempotency: PK key, fingerprint, status, response
booking_id is a remote ID, not a foreign key.
Only successful delivery is recorded.
No cross-database queries or foreign keys.

PK and local FK relations are labeled. Remote references have no cross-database constraints.